

06. FastAPI and Uvicorn

Detailed beginner handbook - English and Arabic

Technologies: FastAPI, Uvicorn

What you will learn

Create a local API route that returns a synthetic summary.

FastAPI maps URLs to Python functions. Uvicorn runs that API server and FastAPI provides /docs automatically.

الهدف: بناء مثال صغير وفهمه خطوة بخطوة.

شرح بسيط: تعلم الفكرة أولاً، ثم جرب المثال الصغير بيانات اصطناعية فقط.

How to study

Study one lesson at a time. Type examples yourself, predict the result, run the code, then change one thing. Keep a notebook of new words, errors, root causes, and fixes.

Course map

Setup and mental model; ten core concepts; a guided mini-project; the real dashboard architecture; debugging; exercises and answers.

Lesson 1 - Setup and mental model

Prerequisites

Windows 10 or 11, VS Code, PowerShell, and permission to install learning dependencies. Use only synthetic data in tutorials and screenshots.

Setup sequence

1. Open PowerShell.
2. Go to learning-examples/06-fastapi-uvicorn.
3. Read README.md.
4. Run `pip install -r requirements.txt; python -m uvicorn main:app --reload`.
5. Read the complete error if it fails.
6. Change one safe value and rerun.

الخطوات: افتح مجلد المثال، شغل الأمر، غير قيمة واحدة، ولاحظ النتيجة.

Mental model

Treat FastAPI, Uvicorn as one layer in a system. Identify the input, the transformation or rule, the output, and possible failures. Understanding that flow is more valuable than memorising syntax.

Checkpoint

Explain: What problem does this technology solve? What is its input? What output should it produce? What can fail?

Lesson 2 - HTTP requests, responses, methods, paths, and status codes

What is it?

HTTP clients send a method, path, headers, and optional body. Servers return status, headers, and body. GET reads; POST submits; 2xx succeeds; 4xx is client error; 5xx is server failure.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In FastAPI and Uvicorn, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
GET /api/summary HTTP/1.1
Host: 127.0.0.1:8000
```

```
# Response: 200 OK
{"total":12}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates HTTP requests, responses, methods, paths, and status codes.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.
هذا الدرس يشرح: HTTP requests, responses, methods, paths, and status codes

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 3 - Creating a FastAPI app and running it with Uvicorn

What is it?

FastAPI holds route definitions. Uvicorn is the ASGI server that accepts connections and calls the app. The module:variable notation `main:app` imports `app` from `main.py`.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In FastAPI and Uvicorn, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
from fastapi import FastAPI
app = FastAPI()
@app.get('/health')
def health(): return {'status':'ready'}
# Run: uvicorn main:app --reload
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Creating a FastAPI app and running it with Uvicorn.

Creating a FastAPI app and running it with Uvicorn: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 4 - Path parameters, query parameters, and request bodies

What is it?

Path parameters identify a resource, query parameters modify a request, and request bodies carry structured data. Type hints drive parsing, validation, and generated documentation.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In FastAPI and Uvicorn, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
@app.get('/records/{record_id}')
def record(record_id: int, details: bool=False):
    return {'id':record_id, 'details':details}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Path parameters, query parameters, and request bodies.

Path parameters, query parameters, and request bodies: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 5 - Pydantic models, validation, serialization, and schemas

What is it?

Pydantic models declare fields and constraints. FastAPI converts valid input to model instances and returns a detailed 422 response when validation fails.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In FastAPI and Uvicorn, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
from pydantic import BaseModel, Field
class Query(BaseModel):
    month: str
    limit: int = Field(10, ge=1, le=100)
@app.post('/query')
def query(body:Query): return body
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Pydantic models, validation, serialization, and schemas.

Pydantic models, validation, serialization, and schemas: هذا الدرس يشرح: حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 6 - GET, POST, file upload, and multipart form data

What is it?

Use GET for safe retrieval and POST for submitted data. UploadFile streams uploaded content and exposes metadata; multipart handling is required for browser file forms.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In FastAPI and Uvicorn, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
from fastapi import UploadFile, File
@app.post('/upload')
async def upload(file:UploadFile=File(...)):
    return {'name':file.filename}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates GET, POST, file upload, and multipart form data.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح: GET, POST, file upload, and multipart form data

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Exceptions, status codes, middleware, and CORS

What is it?

HTTPException produces deliberate client errors. Middleware wraps every request. CORS controls which browser origins may call the API; narrow it to known local origins.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In FastAPI and Uvicorn, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
from fastapi import HTTPException
if not found:
    raise HTTPException(404, 'Record not found')
# Configure CORS only for known frontend origins.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Exceptions, status codes, middleware, and CORS.

Exceptions, status codes, middleware, and CORS: اكتب المثال بنفسك، حدد المدخلات والعمليات والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 8 - Dependency injection and separating routes from business logic

What is it?

Dependencies provide shared validation, authentication, or resources. Keep route functions thin and move calculations into testable service modules.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In FastAPI and Uvicorn, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
from fastapi import Depends
def page_size(limit:int=10): return min(limit,100)
@app.get('/rows')
def rows(limit:int=Depends(page_size)): return []
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Dependency injection and separating routes from business logic.

Dependency injection and separating routes from business logic: هذا الدرس يشرح: حدد المدخلات والعمليات والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 9 - Async versus sync endpoints and blocking work

What is it?

async def can await non-blocking I/O; normal def is suitable for synchronous work. CPU-heavy data processing still blocks and may require a worker strategy.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In FastAPI and Uvicorn, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
@app.get('/wait')
async def wait():
    result = await asynchronous_io()
    return result
# CPU-heavy work is not made non-blocking by async.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Async versus sync endpoints and blocking work.

Async versus sync endpoints and blocking work: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 10 - Automatic OpenAPI documentation and manual API testing

What is it?

FastAPI publishes OpenAPI at /openapi.json and interactive Swagger UI at /docs. Exercise routes with normal, empty, invalid, and oversized requests.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In FastAPI and Uvicorn, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
# Start server, then open:
http://127.0.0.1:8000/docs
# Try valid and invalid values and compare response status/details.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Automatic OpenAPI documentation and manual API testing.

Automatic OpenAPI documentation and manual API testing: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 11 - Security, upload limits, local-only hosting, and production checks

What is it?

Limit upload size and file type, distrust filenames and workbook structure, bind local tools to 127.0.0.1, avoid detailed internal errors, and test security boundaries.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In FastAPI and Uvicorn, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
# Local desktop binding
uvicorn main:app --host 127.0.0.1 --port 8000
# Validate file size/type/content; do not trust filename alone.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Security, upload limits, local-only hosting, and production checks.

هذا الدرس يشرح: Security, upload limits, local-only hosting, and production checks. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Guided mini-project

Project goal

Create a local API route that returns a synthetic summary.

Starting example

```
from fastapi import FastAPI
app=FastAPI()
@app.get('/summary')
def summary(): return {'total':12, 'source':'synthetic'}
```

Read the code

Find imported tools or page elements. Identify the synthetic input. Find the function, event, route, or transformation that changes it. Finally, locate where the result is displayed, returned, saved, or packaged.

Build sequence

1. Run the untouched example.
2. Record the result.
3. Rename one unclear value.
4. Add a synthetic record.
5. Validate empty input.
6. Validate incorrect input.
7. Add a helpful error.
8. Rerun every case.
9. Compare with exercise-solution.
10. Explain every line.

Definition of done

Normal, empty, and invalid cases behave deliberately; no private data is used; and you can explain every line without reading the guide.

Lesson 8 - The real dashboard

Architecture connection

The application combines FastAPI, Uvicorn with the rest of the stack. The frontend gathers filters and files; the local API validates requests; the data layer transforms workbook rows; charts present aggregates; export tools create files; and the desktop layer packages the application for Windows.

Trace the upload feature

The user selects a workbook. The frontend sends it to the local API. The backend validates the file and sheets. The data layer creates safe tables. The API returns metadata. React updates state. Plotly displays results. Identify exactly where this guide's technology participates.

Privacy and quality

Do not place narrative, contact, or identifying fields in tutorials, logs, screenshots, tests, or exports. Use generated identifiers and synthetic totals. Validate at each boundary and collect only fields required for the calculation.

Architecture exercise

Draw five boxes: UI, API, Data Processing, Export, Desktop Packaging. Add arrows and label the data. Circle the box related to this guide and add two validation checks.

Lesson 9 - Debugging

Reliable debugging loop

1. Reproduce with the smallest input.
2. Read the first meaningful error.
3. Find the exact file and line.
4. Inspect value and type.
5. Compare expected and actual results.
6. Change one thing.
7. Rerun the same case.
8. Add a regression test.

Common beginner mistakes

- Wrong folder or command.
- Missing dependency or inactive environment.
- Misspelled file, variable, field, route, or import.
- Assuming input is always present.
- Mixing setup, processing, and display in one block.
- Hiding an exception rather than understanding it.
- Using real sensitive data in a test.

أخطاء شائعة: تشغيل الأمر من مجلد خاطئ، نسيان تثبيت التبعيات، أو تغيير أشياء كثيرة بمرة واحدة.

Error notebook

Record: command, expected result, actual error, root cause, smallest fix, and test added. This turns errors into reusable knowledge.

Lesson 10 - Exercises and answers

Exercises

1. Explain the technology in three sentences.
2. Recreate the example without copying.
3. Add one normal synthetic record.
4. Handle empty input.
5. Handle invalid input.
6. Improve unclear names.
7. Add or describe one test.
8. Locate the technology in the dashboard.
9. Record one error and root cause.
10. Add one mini-project feature.

تمرين: أضف شهرا أو قيمة أمانة جديدة، ثم اشرح ما حدث بكلماتك.

Answer guide

A strong solution is observable and explainable: the documented command works; output changes for the new record; empty and invalid cases have deliberate results; names communicate purpose; a test states an expected result; and the architecture explanation identifies the correct boundary.

Next step

Next: 07. Excel Files with OpenPyXL